

Project – NEAT Application in Racing Car Game

1st Gonalo Vieira

Artificial Intelligence in Games

Faculdade de Ci4ncias da Universidade de Lisboa

Group 18

fc60446@alunos.ciencias.ulisboa.pt

2nd Rodrigo Lopes

Artificial Intelligence in Games

Faculdade de Ci4ncias da Universidade de Lisboa

Group 18

fc59856@alunos.ciencias.ulisboa.pt

Index Terms—NEAT, pygame, Waypoints, Radar, Neural Networks, Genetic Algorithms, Gaussian Noise

I. INTRODUCTION

In this project, we seek to modify a previously developed game (Racing Cars) to encompass the generation of Neural Networks of variable topology. In this game, two cars are placed at the start of a racing track. The purple car has built-in autonomous control, whereas the red car is manually controlled. Upon pressing a key, the player may race the autonomous car until the end of the lap. The cars always stay within the lane due to the presence of impassable walls, defined through a track mask, and as such, the grass is purely cosmetic.

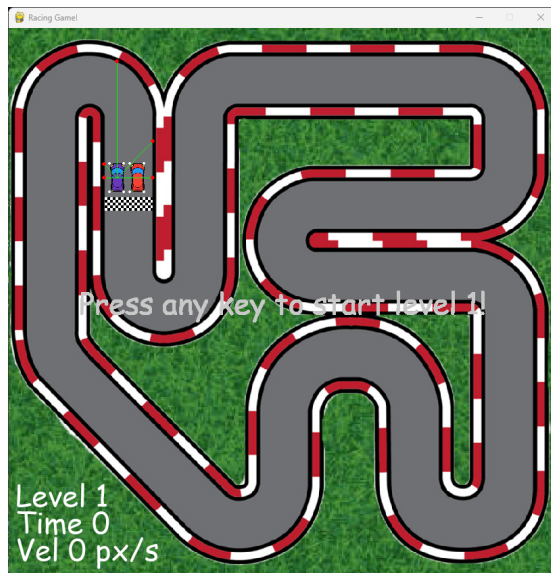


Fig. 1: Original Interface

The automated car has four sensors at each corner of its sprite, which detect when the car is right about to hit a wall. In practice, these are very limited for proper steering, as they do not anticipate the format of the road and only function at the edge of the track. Additionally, their actions are discrete, merely changing rotation or velocity according to the provided values of the sensors. As such, beating the best time of the purple car is seen as a rather easy task. Thus, we seek to use Neural Networks in this project to replace the

current autopilot. This generation will be done through the use of NEAT (NeuroEvolution of Augmenting Topologies), which resorts to genetic algorithms to adapt weights and form connections. These networks will be responsible for guiding the car autonomously towards the finish line under two different navigation systems:

- Waypoints:** Several waypoints are to be manually or procedurally added into the racing track. Information regarding these waypoints is to then be fed to the network, such as the distance or angle relative to a waypoint. There are no restrictions regarding the data that is to be used, as long as it is relative to the inserted waypoints.
- Radars:** The racing car has a finite amount of radar sensors attached to it, which may identify the distance and angle of the car towards walls. The detection distance of these sensors has no limit, but gaussian noise must be implemented on these radars to complicate the learning process. The outputs of the network must be the angular acceleration and speed (magnitude) acceleration for the car.

In both cases, the ultimate goal is to learn the appropriate steering process to complete one lap in the slightest amount of time, under the same racing conditions. As the player car was optionally controlled through the use of waypoints or arrow keys, both navigation systems were already implemented.

II. WAYPOINT LEARNING

For the waypoint implementation, the automated car is guided through a sequence of points manually inserted by the user before training begins. These points are drawn directly on the track using the mouse, and the selected sequence is saved so that the final trained network may later be replayed under the same conditions. This approach differs from radar learning because the car is not required to infer the shape of the road only from wall distances. Instead, it receives explicit information about the next relevant target in the circuit, allowing the network to learn how to steer between successive reference points.

The original waypoint-based controller used in laboratory classes relied on fixed decision rules. The car computed the angle to the current waypoint and then selected discrete actions such as turning left, turning right, or moving forward. In this project, this rule-based logic was replaced by a NEAT-evolved neural controller. The waypoint list is still used as a navigation

guide, but the decision of how much to rotate and accelerate is now produced by the evolved network.

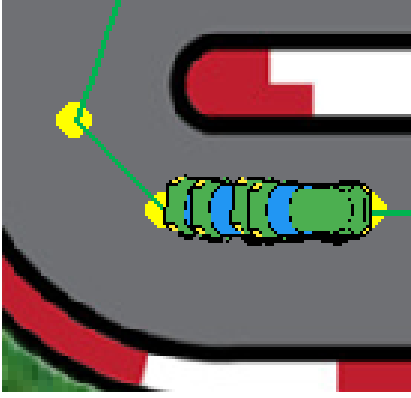


Fig. 2: Waypoint navigation setup, where the yellow dots represent the manually selected path

The network receives five normalized inputs. The first input is the signed angle between the car's current orientation and the current waypoint, normalized to the range $[-1, 1]$. The second input is the Euclidean distance to the current waypoint, normalized according to the maximum possible distance in the game window. The third input represents the angle between the current waypoint segment and the following segment, giving the network information about the upcoming curve. This is useful because it allows the car to begin adjusting before reaching the waypoint itself. The fourth input is the current velocity of the car, normalized by the maximum velocity, and the fifth input is the current progress through the waypoint list.

As with the radar controller, the neural network has two outputs. The first output controls angular acceleration, while the second controls speed acceleration. Both outputs use the $\tanh$ activation function and are therefore bounded between $[-1, 1]$. These values are then scaled according to the car's physical limits, namely its maximum rotation velocity and acceleration. This keeps the neural controller compatible with the original game physics, instead of allowing the network to perform physically impossible turns or speed changes.

A. Testing Methodology

The NEAT configuration used for the waypoint controller follows the same principles as the radar approach, but with input values adapted to waypoint geometry rather than radar distances. Table I summarizes the most relevant configuration parameters.

Each generation was evaluated for a maximum of 1800 frames. During this period, every genome controlled one car from the initial position. The car attempted to reach the waypoint sequence in order, and a waypoint was considered reached when the distance between the car and the waypoint became smaller than 35 pixels. Once this happened, the next waypoint became the new target. This process continued until

the car completed the full waypoint list, collided with the track border, stalled, or reached the frame limit.

The fitness function was designed to reward both navigation progress and smooth forward movement:

- a) **Velocity:** At each frame, the car receives a small reward proportional to its current velocity. This encourages forward movement while discouraging reversing.
- b) **Waypoint Approach:** The car receives fitness when it reduces its distance to the current waypoint. This rewards movement in the correct direction even before the waypoint is fully reached.
- c) **Waypoint Completion:** A large bonus is awarded whenever a waypoint is reached, increasing with the number of waypoints already completed.
- d) **Full Route Completion:** Completing the full waypoint sequence awards an additional bonus, with faster completion receiving a higher reward.
- e) **Collision Penalty:** Cars that collide with the track border are penalized and removed from the current generation.
- f) **Stall Penalty:** Cars that fail to move at least 20 pixels within a 90-frame window are removed, avoiding behaviours where the network oscillates in place.

The resulting fitness function is therefore more goal-directed than the radar one. Instead of rewarding only lap completion and general speed, the waypoint controller receives intermediate feedback throughout the route. This makes early learning easier, since a car does not need to complete a full lap before receiving meaningful reward.

B. Waypoint Persistence and Replay

To ensure that the evaluation phase uses the same path as training, the selected waypoint list is saved immediately when training begins. The best waypoint genome is also saved after every generation. This is important because training may be interrupted before all generations are completed, and without intermediate saving the replay system would keep using an older waypoint sequence or an older network.

The final trained waypoint controller can then be compared against the radar controller in a duel mode. In this mode, the waypoint car uses the saved `winner_waypoints.pkl` genome and the saved `waypoints.pkl` route, while the radar car uses the saved `winner_radar.pkl` genome. This allows both navigation approaches to be tested under the same track conditions, although the comparison must be interpreted carefully: the waypoint car is explicitly trained to follow the selected waypoint sequence, whereas the radar car is trained to reach the finish line through local wall sensing.

C. Discussion

The waypoint approach offers a more direct representation of the desired path and therefore tends to learn meaningful behaviour faster than the radar approach. Since the network knows both where the next target is and how the following segment turns, it can anticipate curves rather than reacting only when walls become close. This is especially useful in sections of the track where radar readings may be ambiguous

TABLE I: Waypoint NEAT Configuration Parameters

Parameter	Value	Reasoning
pop_size	50	Maintains reasonable diversity without excessive training time
num_inputs	5	Angle, distance, next curve, speed and progress
num_outputs	2	Δ angle and Δ speed
num_hidden	0	NEAT grows complexity only if needed
activation	tanh	Naturally bounds output values in $[-1, 1]$
feed_forward	True	Frame reaction does not require recurrent memory
initial_connection	full	Every input influences both outputs from generation 0
fitness_criterion	max	Selects the genome with highest fitness
reset_on_extinction	True	Prevents training from stopping after full species extinction

or where the car must begin turning before a wall is directly in front of it.

However, the waypoint method also depends heavily on the quality of the selected points. If the waypoints are too sparse, the car may cut corners too aggressively and collide with the track border. If they are too dense, the car may over-correct between nearby targets, causing jitter or unnecessary braking. Therefore, waypoint placement acts as a form of prior knowledge provided by the user. A well-designed waypoint sequence can greatly simplify the learning problem, while a poor sequence can make even a good neural controller appear ineffective.

Overall, waypoint learning proved to be a useful alternative to radar-based navigation. It provides stronger intermediate feedback and gives the neural network explicit information about the intended route. On the other hand, its performance is less general, as changing the route or track direction requires redefining the waypoint sequence and retraining or replaying the controller under the new path.

III. RADAR LEARNING

For our radar implementation, we chose to provide the automated car with 5 radar sensors, as suggested by the project statement. These sensors are angled at $[-90, -45, 0, 45, 90]$ degrees relative to the center of the car, assuring a simple and comprehensive understanding of both the sides and front of the track. Additionally, the equal distance difference between the sensors negates any bias relative to either sensor.

The distances provided by each of these sensors are then provided to NEAT as inputs, normalized between 0 and 1. This normalization process, known as min-max scaling, aids the Neural Network to converge, stopping large-scale features from dominating the learning process. The network then uses tanh as an activation function for its outputs. This decision enforces outputs between -1 and 1, ideal for rotation and speed bounds.

A. Testing Methodology

Prior to training, all NEAT hyper-parameters were fixed through a combination of theoretical reasoning and empirical testing. The full configuration is presented in Table II, alongside the chosen values and the rationale behind each choice.

With these parameters established, generation count and population size were identified as the two remaining variables

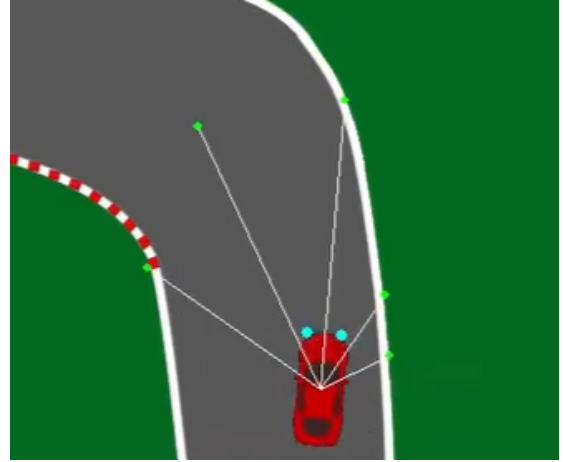


Fig. 3: Visualization of the implemented radars
The distance measurement limit is of 200 pixels

most likely to influence convergence, as they directly govern the diversity and duration of the evolutionary search. These two parameters were then systematically varied across nine runs as shown in Table III, allowing a direct comparison of their individual and combined effects on training performance and time taken. To achieve reasonable training time for the scope of this project whilst maximizing optimizations, object rendering was completely disabled, allowing the game to run at very high, uncapped FPS.

TABLE III: Test Run Configuration

	50 gens	100 gens	150 gens
50 pop	Run 1	Run 2	Run 3
100 pop	Run 4	Run 5	Run 6
150 pop	Run 7	Run 8	Run 9

where $gens/pop = generations/population$

Naturally, for our Neural Network to function, we had to define what actions or outcomes provided higher fitness, as follows:

- Car Velocity:** Each car is awarded fitness points according to their value of velocity each frame, which varies between $[-2, 4]$. As such, backwards movement counts as a penalty.
- Lap Completion:** A flat base bonus of 1000 points was given to any car capable of completing one lap. An

TABLE II: NEAT Configuration Parameters

Parameter	Value	Description	Reasoning
<i>General</i>			
fitness_criterion	max	Winner is the highest fitness genome	Select the best car
reset_on_extinction	True	Restart if all species die out	Prevents crashes on full extinction
<i>Network</i>			
num_inputs	5	One per radar	Simple and effective with min-max scaling
num_outputs	2	Δ angle, Δ speed	Forced by project specification
num_hidden	0	Automatically grown by NEAT	Complexity grown only when beneficial
activation	tanh	Outputs in $[-1, 1]$, scaled to car limits	Naturally bounded
feed_forward	True	No recurrent connections	Car reaction doesn't depend on sensor history
initial_connection	full	Inputs and outputs fully connected	Every radar influences both outputs from the start
<i>Weights & Biases</i>			
weight_mutate_rate	0.6	Probability of nudging a weight	Cars already solve the track, focus on refinement
weight_mutate_power	0.3	Std. dev. of weight nudge	Preserves working solutions and allows fine-tuning
weight_replace_rate	0.1	Probability of full weight replacement	Introduces minor diversity
weight_init_stdev	1.0	Initial weight spread	Sufficient for varied behaviours in generation 0
bias_init_mean	0	Initial mean of the bias	0 at generation 0, adapted over time
bias_init_stdev	1.0	Std. dev. of the bias	Some spread so early networks aren't all identical
bias_mutate_rate	0.5	Probability of nudging a bias per generation	Biases are for corrections, not the primary signal
bias_mutate_power	0.3	Std. dev. of the nudge applied to a bias	Matched to weight power
<i>Topology Mutation</i>			
node_add_prob	0.1	Probability of inserting a hidden node	Low as problem is simple enough
node_delete_prob	0.2	Probability of removing a hidden node	Encourages pruning of redundant structure
conn_add_prob	0.3	Probability of adding a connection	Network is richer but not complex early on
conn_delete_prob	0.3	Probability of removing a connection	Symmetric with add
<i>Speciation</i>			
compatibility_threshold	3.0	Distance threshold for species separation	Default value
compatibility_disjoint	1.0	Weight of structural differences	Default value
compatibility_weight	0.5	Weight of weight differences	Topology differences matter more
<i>Stagnation & Reproduction</i>			
max_stagnation	15	Generations before a stagnant species is killed	Redirects resources to improving species faster
species_elitism	2	Best species always preserved	Self-explanatory
elitism	3	Top genomes per species carried unchanged	Protects best solutions from mutation
survival_threshold	0.25	Top 25% of each species reproduce	Slightly higher than default, greedy approach

exponential time bonus is also awarded for faster laps, such that saving a second at a fast time is worth more than saving a second at a slow time.

$$t_{bonus} = 5000 * \exp(-frame/400)$$

- c) **Fitness Floor:** As NEAT expects non-negative values, all negative fitness values were clamped to zero.

Additionally, we also imposed several constraints on each car to ensure fair and meaningful evaluation:

- a) **Frame Cap:** Each generation was capped at 1500 frames (a large enough amount at which completed laps are irrelevant), after which all surviving cars were removed and the generation ended. This ensures generations do not continue endlessly.
- b) **Anticheat:** We quickly noted that cars were rotating towards the finish line right at the start for the huge fitness bonus. Naturally, as this behavior is unintended, we killed cars that attempted to do this instead.
- c) **Stall Detection:** Eliminates any car that failed to move at least 30 pixels within a 60-frame window, preventing cars from oscillating in place and stalling runs.

Car physics were fixed across all runs, with a maximum velocity of 4 pixels per frame, a rotation velocity of 4 degrees per frame, and an acceleration of 1 pixel per frame squared.

B. Original Track

Each of the nine training runs was replayed three times to obtain a representative lap time. This was necessary because Gaussian noise applied to both sensor readings and actuator outputs, as required by the project specification, which occasionally made the car fail to clear a lap. As such, the average of the three recorded lap times (in milliseconds) is used as the final performance metric for each run.

TABLE IV: Training and Replay Results

Run	Pop	Gens	Train (s)	Best Fitness	Avg Lap $\pm$ Std (ms)
Run 1	50	50	29.47	3896	15845 $\pm$ 57
Run 2	50	100	142.12	3950	16161 $\pm$ 73
Run 3	50	150	302.17	3982	16353 $\pm$ 106
Run 4	100	50	88.13	3909	16113 $\pm$ 56
Run 5	100	100	274.59	4005	16293 $\pm$ 5
Run 6	100	150	459.65	3990	16364 $\pm$ 79
Run 7	150	50	193.94	3938	16293 $\pm$ 107
Run 8	150	100	528.87	3990	16389 $\pm$ 72
Run 9	150	150	831.08	4002	16469 $\pm$ 61

Total training time across all nine runs was approximately 47.5 minutes. As expected, training time scales directly with the product of population size and generation count — pop50_gen50 completed in under 30 seconds, while pop150_gen150 required over 13 minutes. A more significant finding is that population size had a stronger influence on convergence speed than generation count: runs with 50

genomes required between 6 and 28 generations to produce the first valid lap completion, whereas runs with 100 or 150 genomes consistently achieved this within 5 to 9 generations. Beyond the first lap, however, additional generations yielded diminishing returns. The best average lap time was achieved with the least amount of genomes and generations, with minimal differences across different runs. The best fitness scores converged to a narrow range of approximately 3896 to 4005, suggesting the fitness function ceiling was reached regardless of the configuration. As a result, it becomes clear that gaussian noise by itself, responsible for the deviation across runs, plays a greater impact in run time than additional generations or genomes. This motivates the introduction of a corner quality reward in the second set of runs, as recommended in the project statement, which aims to raise that ceiling.

C. Curve Optimization

As stated in section 4 of the project statement, it would be ideal for cars to perform curves from the inside-out, as seen in real competitive racing such as Formula 1. Despite numerous attempts, unfortunately, this behavior could not be implemented properly with just 5 sensors. The car's maximum rotation velocity of 4 degrees per frame, a constraint inherited from the original Racing Car game which cannot be altered, stops any attempts at performing aggressive curves, which paired with the weak curvature detection makes it physically impossible for the car to rotate in time for some of the shorter curves.

Nonetheless, with the use of 9 sensors $[-90, -45, -30, -15, 0, 15, 30, 45, 90]$ we could get a consistent performance boost of about half a second, detailed in Table V. By awarding a small fitness boost of 5 points whenever either $\pm 45^\circ$ diagonal sensor (indices 1 and 7) drops below 60 pixels in one frame and spikes above 120 pixels in the next, the car can detect that the inner wall of a curve just opened up on that side, signaling the optimal turn-in point. Unfortunately, the physics of the original game still heavily limits the performance gains from these improvements, as turning faster would naturally improve lap times.

TABLE V: Training and Replay Results

Run	Pop	Gens	Train (s)	Best Fitness	Avg Lap $\pm$ Std (ms)
Run 1	50	50	131	3987	15325 $\pm$ 46
Run 2	50	100	319	3973	15535 $\pm$ 37
Run 3	50	150	255	4206	16982 $\pm$ 17
Run 4	100	50	289	4063	16271 $\pm$ 52
Run 5	100	100	688	4012	15709 $\pm$ 55
Run 6	100	150	782	4239	16902 $\pm$ 60
Run 7	150	50	413	3973	15672 $\pm$ 32
Run 8	150	100	999	4164	16732 $\pm$ 7
Run 9	150	150	1157	4156	16617 $\pm$ 46

The training time under the same conditions as beforehand with the curving rewards is of 84 minutes, a large difference from the previous 45 minutes. This is a direct result of quicker convergence, with cars performing laps much sooner. However, we note that, as seen beforehand, a high number of generations or population leads to diminishing returns. This is

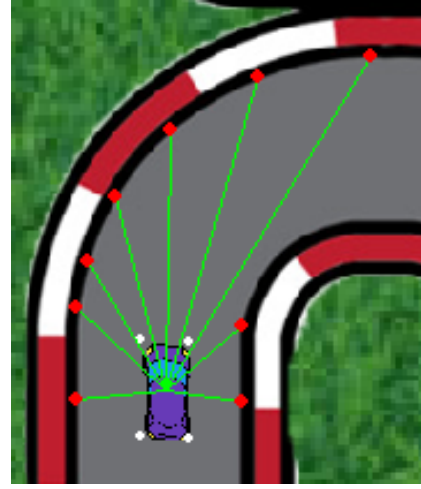


Fig. 4: Configuration with 9 sensors, demonstrating the impact of the newly added 45° and 30° degree sensors

a direct result of overfitting, where cars begin to exploit the fitness function by curving constantly throughout the track. With the best performance being a result of such a low number of generations and population, we also gain an advantage in having such a low training time for optimal results.

As the best possible lap time is thought to be of around 15.0 milliseconds, we consider these results to be a success.

D. Reverse Track

Testing on a reversed track or an alternative finish line was not pursued, as the agent's behaviour is driven entirely by local sensor readings relative to the car's heading — it has no hardcoded knowledge of track geometry or direction. A reversed layout would present the same sequence of sensor patterns in the same order, and an alternative finish line would only alter when the lap-completion bonus fires, not how the car navigates, therefore it would be redundant to do so. Given that the fitness function already rewards forward progress and penalizes walls independently of any fixed landmark, the results would be structurally identical to those reported. As such, we chose to prioritize the improvement of scores across the previous sections.

IV. FINAL COMPARISON AND CONCLUSION

Both approaches successfully trained autonomous agents capable of completing the track, but they differ substantially in robustness, effort, and adaptability. In an attempt to determine which navigation method produced the best possible time, two cars were put within the same track at the same starting position. The waypoint navigation and radar navigation were given to the green and purple car, respectively.

We determined that the waypoint-guided agent is highly sensitive to the introduced Gaussian noise as a result of extremely tight, difficult curves. In practice, only approximately 1 in 500 runs complete a clean lap. The radar-based agent, by contrast, reacts to its local sensor readings every frame



Fig. 5: Both navigation systems racing simultaneously

and continuously corrects its path, making it effectively noise-resistant, with failures occurring negligibly, almost always due to an unlucky burst of noise rather than a structural weakness in the approach.

In terms of raw lap time, the radar agent narrowly edges out the waypoint agent by less than half a millisecond, but not within noise variance. Theoretically, a waypoint layout with pixel-perfect waypoint placements would yield a faster time by enforcing an perfected racing line. However, finding such a layout in practice is extremely tedious: waypoints must be placed, trained, evaluated, and iteratively adjusted by hand, with no guarantee of convergence to an optimal configuration.

The radar agent requires none of this manual intervention. It adapts to any track geometry automatically, adapts its racing line through evolution rather than human tuning and performs just as fine. Given the marginal performance gap and the substantial difference in reliability and development effort, the radar-based approach is the clearly superior solution for this problem. Furthermore, the impact of mutation probability and other hyper-parameters within the learning process is very minimal, as convergence occurs within very few generations in both navigation systems.